

Autodesk® Scaleform®

HUD

Scaleform 4.3 HUD

Flash UI

C++

AAA

Nate Mitchell, Prasad Silva

2.0

2010 7 30

Copyright Notice

Autodesk® Scaleform® 4.3

© 2013 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and other countries: 123D, 3ds Max, Algor, Alias, AliasStudio, ATC, AutoCAD, AutoCAD Learning Assistance, AutoCAD LT, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk, Autodesk 123D, Autodesk Homestyler, Autodesk Intent, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSketch, AutoSnap, AutoTrack, Backburner, Backdraft, Beast, Beast (design/logo), BIM 360, Built with ObjectARX (design/logo), Burn, Buzzsaw, CADmep, CAiCE, CAMduct, CFdesign, Civil 3D, Cleaner, Cleaner Central, ClearScale, Colour Warper, Combustion, Communication Specification, Constructware, Content Explorer, Creative Bridge, Dancing Baby (image), DesignCenter, Design Doctor, Designer's Toolkit, DesignKids, DesignProf, Design Server, DesignStudio, Design Web Format, Discreet, DWF, DWG, DWG (design/logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DWGX, DXF, Ecotect, ESTmep, Evolver, Exposure, Extending the Design Team, FABmep, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, ForceEffect, Freewheel, GDX Driver, Glue, Green Building Studio, Heads-up Design, Heidi, Homestyler, HumanIK, i-drop, ImageModeler, iMOUT, Incinerator, Inferno, Instructables, Instructables (stylized robot design/logo), Inventor, Inventor LT, Kynapse, Kynogon, LandXplorer, Lustre, Map It, Build It, Use It, MatchMover, Maya, Mechanical Desktop, MIMI, Moldflow, Moldflow Plastics Advisers, Moldflow Plastics Insight, Moondust, MotionBuilder, Movimento, MPA, MPA (design/logo), MPI (design/logo), MPX, MPX (design/logo), Mudbox, Multi-Master Editing, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, Pixlr, Pixlr-o-matic, PolarSnap, Powered with Autodesk Technology, Productstream, ProMaterials, RasterDWG, RealDWG, Real-time Roto, Recognize, Render Queue, Retimer, Reveal, Revit, Revit LT, RiverCAD, Robot, Scaleform, Scaleform GfX, Showcase, Show Me, ShowMotion, SketchBook, Smoke, Softimage, Socialcam, Sparks, SteeringWheels, Stitcher, Stone, StormNET, TinkerBox, ToolClip, Topobase, Toxik, TrustedDWG, T-Splines, U-Vis, ViewCube, Visual, Visual LISP, Vtour, WaterNetworks, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. "AS IS." AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

Autodesk Scaleform

HUD

Autodesk Scaleform Corporation

6305 Ivy Lane, Suite 310

Greenbelt, MD 20770, USA

www.scaleform.com

info@scaleform.com

(301) 446-3200

(301) 446-3199

1		1
2		2
2.1	(Build)	2
2.2		3
2.3		3
2.3.1	(Windows).....	3
2.3.2	(Xbox/PS3)	4
3		5
3.1	C++.....	5
3.1.1		5
3.1.2	HUD/	5
3.1.3		6
3.2	Flash	6
3.2.1	HUDKit.flas	6
3.2.2	Minimap.flas	7
4 HUD		10
4.1		10
4.2	Player Stats Ammo	12
4.3		16
4.4		17
4.5		19
4.6		21
4.7		21
4.8		22
4.9		23

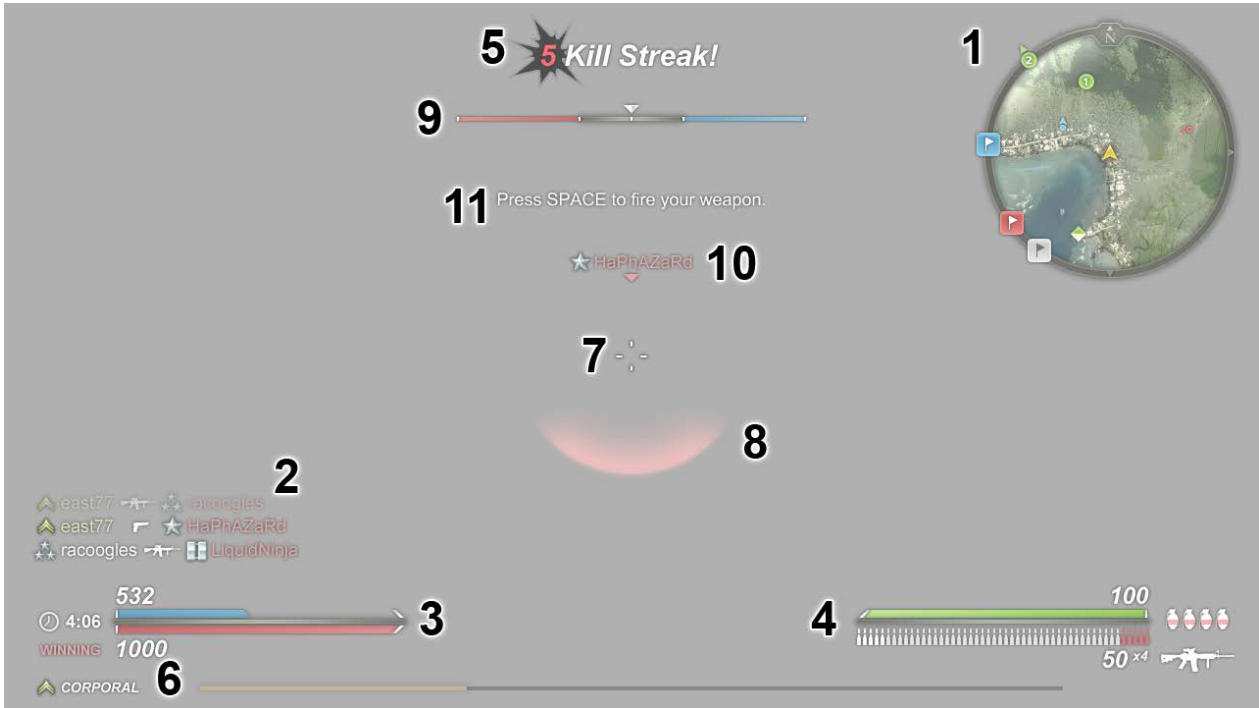
4.10		25
5		29
5.1.1		29
6		31
6.1		31
6.2		32

1

Scaleform HUD

Scaleform® Direct Access API

AAA
HUD
(FPS) UI



1 HUD

HUD

UI

- 1. (Minimap)
- 2.
- 3.
- 4. / /
- 5.
- 6.
- 7.
- 8.
- 9.
- 10. /
- 11.

FPS

2

2.1 (Build)

- *Apps\Kits\HUD* - HUD C++
- *Bin\Data\AS2 or AS3\Kits\HUD* - Flash ActionScript AS2 ActionScript 2
Flash 8 AS3 ActionScript 3 Flash 10
- *Projects\Win32 \{Msvc80 or Msvc90}\Kits\HUD –* Windows Visual Studio
2005/2008
- *Projects\Xbox360\{Msvc80 or Msvc90}\Kits\HUD –* Xbox 360 Visual Studio
2005/2008
- *Projects\Common\HudKit.mk –* HUD PS3

Bin\Kits\HUD Windows HUDKit.exe
Scaleform SDK

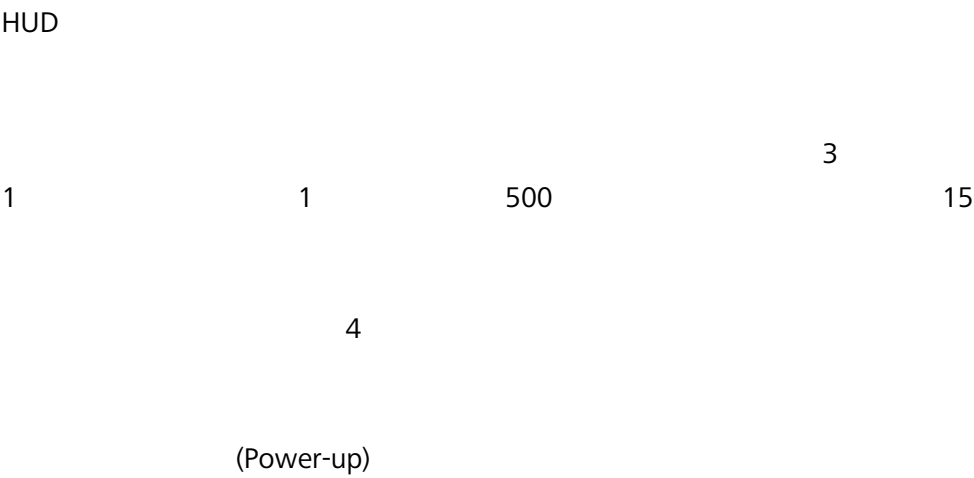
Windows *Projects\Win32\Msvc80\Kits Msvc90\Kits* Scaleform 4.3 Kits.sln
Debugging Working
Bin\Data\AS2\Kits\HUD or Bin\Data\AS3\Kits\HUD

Xbox 360 *Projects\Xbox360\{Msvc80 Msvc90}\Kits\HUD* Scaleform 4.3 Kits.sln
Bin\Data\AS2 or
AS3\Kits\HUD Xbox 360

PS3 Scaleform *make* HUD
(HUD Kit) PS3 SN Systems .
Bin\PS3

o app_home/ Directory: *{Local Scaleform Directory}\Bin\Data\AS2 or AS3\Kits\HUD*

2.2



2.3

2.3.1 (Windows)

W, A, S, D	WSA, D
SPACE	
R	
G	
ESC	(Text Field)
B	AI
1, 2	• 1 • 2

2.3.2 (Xbox/PS3)

	W, S	A, D
X /		
B /		
Y /		

3

HUD Flash UI HUD C++ C++
HUD

3.1 C++

C++ HUD MVC
Flash UI Fx HUD (View)

HUD HUD HUD HUD

UI Scaleform Scaleform Direct Access API

C++ *Apps\Kits\HUD*

3.1.1

- **HUDKitDemo.cpp** – Scaleform Player
C++ SWF
- **HUDKitDemoConfig.** – FxPlayerConfig.h HUDKitDemo
Windows

3.1.2 HUD/

- **FxHUDKit.h** – HUD HUD
- **FxHUDView.h/.cpp** – HUD (Billboard)
- **FxMinimap.h** –
- **FxMinimapView.h/.cpp** –

3.1.3

- **HUDAdapter.h/.cpp** – HUD
- **HUDEntityController.h/.cpp** – AI
- **HUDSimulation.h/.cpp** – UI

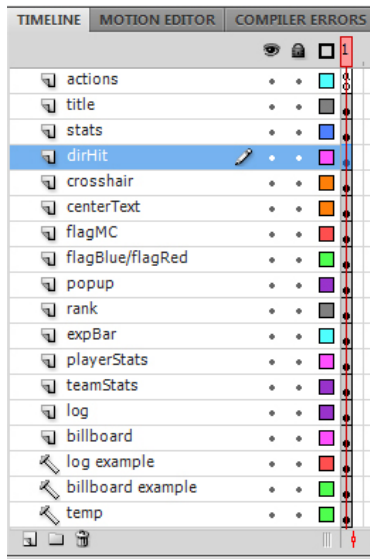
3.2 Flash

HUD	Flash	HUDKit.fl.a	Minimap.fl.a	FLA
HUD	UI	Scaleform	UI	

Scaleform C++ ActionScript Flash Flash HUD
Classic Tweens Flash Gfx::Value::GotoAndPlay()
C++

3.2.1 HUDKit.flas

<i>Bin\Data\AS2 or AS3\Kits\HUD</i>	HUDKit.fla	SWF	HUDKitDemo
Flash UI	HUDKit.fla		



2 HUDKit.fla

HUDKitDemo Frame 1 1 External Interface HUDKit MovieClip

ExternalInterface.call("registerHUDView", this);

MovieClip C++ MovieClip C++ HUDView

HUD MovieClips Scaleform C++

3.2.2 Minimap.fla

3 Minimap

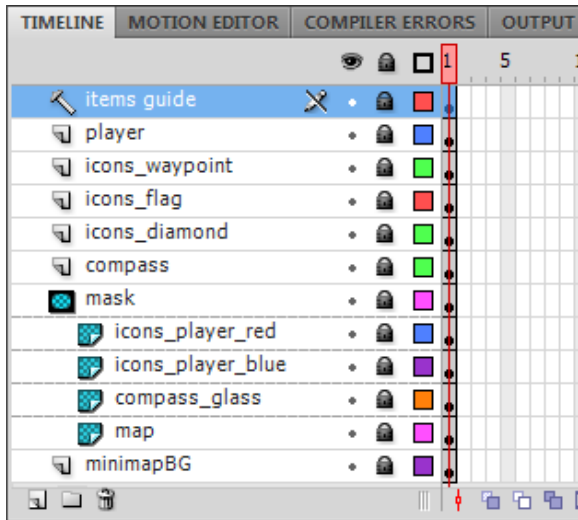
Minimap.fla 'Minimap'
3D

-
-
- (Flag)
-

Direct Access API
C++

MovieClips (Stage) MovieClips

'Minimap'



4 'minimap'

- **items guide** / (Guide Layer)
- **player**
- **icons_waypoint**
- **icons_flag**
- **icons_diamond**
- **compass**
- **mask** (Mask Layer)

SWF

- o icons_player_red
- o icons_player_blue
- o compass_glass
- o map

C++

/

- **minimapBG**

C++

'Minimap'

com.scaleform.Minimap

Bin\Data\AS2 or AS3\Kits\HUD\com\scaleform

HUDKitDemo

4 HUD

C++ HUD

HUDKit.swf

- FxHUDView – HUD HUDKit.swf (Core Manager)
MovieClips
- BillboardCache – /
-
- FxHUDLog – HUD FxHUDMessages
- FxHUDMessage –

FxHUDView

MC MovieClip

FxHUDView

UpdateView()

MovieClip

HUDKit.fla

UI

FxHUDView::UpdateView()

UpdateTeamStats() UpdatePlayerEXP() Update PlayerStats() UpdateEvents() UpdateTutorial()
UpdateBillboards() UI

UpdateView()

Scaleform Direct Access API

Flash

GfX::Value

Scaleform

Flash

GfX::Value::SetDisplayInfo() GfX::Value::GetDisplayInfo() GfX::Value::SetText() HUD

4.1

teamStats MovieClip

HUDKit.fla

1 (Scene 1)

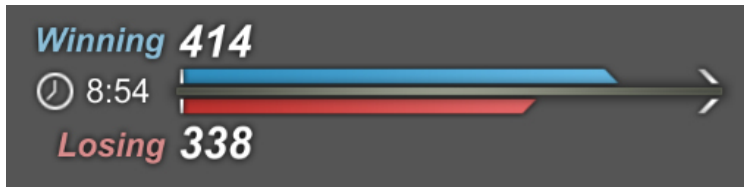
teamStats

(*redWinning, blueWinning*)

(*scoreRed, scoreBlue*)

(*teamRed, teamBlue*)

(*roundTime*)



5 teamStats

FxHUDView::UpdateTeamStats()

teamStats

teamStats MovieClip

```
void FxHUDView::UpdateTeamStats(FxHUDEnvironment *penv)
{
    // Retrieve the scores from the simulation.
    unsigned scoreBlue = unsigned(penv->GetHUDBlueTeamScore());
    unsigned scoreRed = unsigned(penv->GetHUDRedTeamScore());

    String text;
    Value tf;
    Value::DisplayInfo info;

    // We won't update any element who has not changed since the last update
    // to avoid unnecessary updates. To do so, we compare the previous
    // simulation State to the latest simulation data. If the two are different,
    // update the UI element in the HUD View.
    if (State.ScoreRed != scoreRed)
    {
        Format(text, "{0}", scoreRed);
        ScoreRedMC.SetText(text); Update the red team score text.

        info.SetScale((scoreRed / (Double)MaxScore) * 100, 100);
        TeamRedMC.SetDisplayInfo(info); // Update and scale the red team score
                                         bar movieClip.
    }

    if (State.ScoreBlue != scoreBlue)
    {
        Format(text, "{0}", scoreBlue);
        ScoreBlueMC.SetText(text); // Update the blue team score text.

        info.SetScale((scoreBlue / (Double)MaxScore) * 100, 100);
        TeamBlueMC.SetDisplayInfo(info); // Update and scale the blue team
                                         score bar movieClip.
    }

    if (State.ScoreRed != scoreRed || State.ScoreBlue != scoreBlue)
    {
        // Update the "Winning" and "Losing" text fields for both teams
        // if the score has changed.
        if (scoreBlue > scoreRed)
```

```

        {
            RedWinningMC.SetText(LosingText);
            BlueWinningMC.SetText(WinningText);
        }
        else
        {
            RedWinningMC.SetText(WinningText);
            BlueWinningMC.SetText(LosingText);
        }
    }

    // Update the roundtime clock
    float secondsLeft = penv->GetSecondsLeftInRound();
    unsigned sec = ((unsigned)secondsLeft % 60);
    if (sec != State.Sec)
    {
        unsigned min = ((unsigned)secondsLeft / 60);
        String timeLeft;
        Format(timeLeft, "{0}:{1:0.2}", min, sec);
        RoundTimeMC.SetText(timeLeft);
        State.Sec = sec;
    }
}

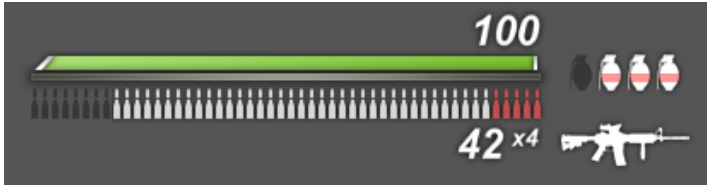
```

	Scaleform::String	GfX::Value::SetText()	<i>scoreRed</i>	<i>scoreBlue</i>
teamRed	teamBlue	X	GfX::Value::SetDisplayInfo()	0%

	<i>blueWinning/redWinning</i>		
GfX::Value::SetText()	HUDView	"Winning"	"Losing"
	textField	UpdateView()	
		GfX::Value::SetText()	
<i>roundTime</i>	textField		

4.2 Player Stats Ammo

<i>playerStats</i>	HUDKit fla	1	<i>playerStats</i>	(<i>healthN</i> ,
<i>health</i>)	(<i>weapon</i>) HUD		(<i>reload</i>)	



6 playerStats

clipN ammoN ammo ammoBG grenade grenadeBG

clipN ammoN

ammo/healthVehicle

ammo ammoBG

51 Keyframes

GfX::Value::GotoAndStop()

- *ammo*
- *ammoBG*

grenade/armor ammo

grenade grenadeBG

5

1 4

5

[GfX::Value::GotoAndStop\(\)](#)

- *grenade*
- *grenadeBG*

playerStats

```
void FxHUDView::UpdatePlayerStats(FxHUDEnvironment *penv)
{
    FxHUDPlayer* pplayer = penv->GetHUDPlayer();

    // Load latest player info.
    unsigned ammoInClip = pplayer->GetNumAmmoInClip();
    unsigned ammoClips = pplayer->GetNumClips();
    unsigned clipSize = pplayer->GetMaxAmmoInClip();
    unsigned grenades = pplayer->GetNumGrenades();

    unsigned ammoFrames = 51;
    unsigned grenadeFrames = 5;

    String text;
```

```

if(ppplayer->GetHealth() != State.Health)
{
    unsigned health = unsigned(ppplayer->GetHealth() * 100);
    Format(text, "{0}", health);
    HealthNMC.SetText(text); // Update the health text field.

    Value::DisplayInfo info;
    info.SetScale(Double(health), 100);
    HealthMC.SetDisplayInfo(info); // Update and scale the health bar.
}

if(ammoInClip != State.AmmoInClip)
{
    Format(text, "{0}", ammoInClip);
    AmmoNMC.SetText(text); // Update the ammo text field.

    // The ammo is divided into two parts: the white bullet icons (AmmoMC)
    // and their respective backgrounds (AmmoBGMC).

    // To update the number of bullets currently in the clip, we use
    // GotoAndStop with AmmoMC and select the frame with the proper number
    // of white bullet icons. There are 51 frames in AmmoMC (defined above
    // in ammoFrames) and the first frame displays every bullet icon.

    // To display X number of bullets, we subtract X from the total number
    // of frames in AmmoMC (51) and GotoAndStop on the result.
    AmmoMC.GotoAndStop(ammoFrames - ammoInClip);
}

if(ammoClips != State.AmmoClips)
{
    Format(text, "x{0}", ammoClips);
    ClipNMC.SetText(text); // Update the remaining ammo clips text field.
}

if(grenades != State.Grenades)
{
    // Grenades are setup similar to Ammo. The first frame for the
    // Grenades movieClip shows 4 white grenade icons. The second frame
    // shows 3.

    // To display X number of grenades, we subtract X from the total number
    // of frames in GrenadeMC (5) and GotoAndStop on the result.
    // Note: GrenadeMC actually contains 10 frames but 6-10 are unused by
    // this implementation.
    GrenadeMC.GotoAndStop(grenadeFrames - grenades);
}

```

```

if (pplayer->GetWeaponType() != State.weaponType)
{
    Change the weapon icon if the player has changed weapons.
    unsigned weaponFrame = GetWeaponFrame(pplayer->GetWeaponType());
    WeaponMC.GotoAndStop(weaponFrame);

    // Update the ammo background icons based on clipSize.
    if (clipSize != State.AmmoClipSize)
        AmmoBGMC.GotoAndStop(ammoFrames - clipSize);
}

// If the ammo in clip is less than 6 bullets, play the "Reload" indicator
// movieClip.
if (ammoInClip <= 5)
{
    // ReloadMC will play until we GotoAndStop on Frame 1, so no need to
    // start the animation more than once.
    if (!bReloadMCVisible)
    {
        bReloadMCVisible = true;
        ReloadMC.GotoAndPlay("on"); // Will cycle back to "on" frame when
                                    // it reaches the end and play again.
    }
}

// If the reload indicator is displayed but there are more than six bullets
// hide it and stop the animation by playing frame 1 of the movieClip.
else if (bReloadMCVisible)
{
    ReloadMC.GotoAndStop("1"); // Frame 1 contains stop();
    bReloadMCVisible = false;
}
}

```

ammo 1 50 51 0

ammoMC.GotoAndStop(ammoFrames - ammoInClip) HUD

ammoBG ammoBGMC.GotoAndStop() X X

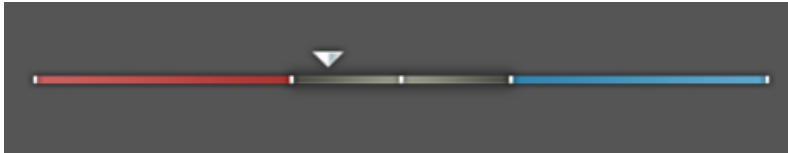
(Machine Gun) 50

ammoBG.GotoAndStop(1) 1 50

FxHUDView::UpdateView() MovieClip

4.3

flagMC HUDKit fla 1 (arrow) *flagMC* *flag*
CaptureState



7 *flagMC*

flagMC Classic Tween Flash *flagMC*
4 Hidden Visible Fade-In
FadeOut GotoAndStop() Keyframe / 1 5 "On" "Off"

FxHUDView::UpdateEvents()

flagMC

arrow

```
// Flag capture indicators and reticule behavior.
void FxHUDView::UpdateEvents(FxHUDEnvironment *penv)
{
    Value::DisplayInfo info;

    if (penv->IsHUDPlayerCapturingObjective())
    {
        if(!bFlagMCVisible)
        {
            SetVisible(&FlagMC, true); // Set the Flag MovieClip's visibility
                                     // to true
            FlagMC.GotoAndPlay("on"); // Play the fade-in animation once.
            bFlagMCVisible = true;
        }

        // Shift the x-coordinate of the Flag Arrow to show the capture state
        // of the flag the player is currently capturing.
        info.SetX(penv->GetHUDCaptureObjectiveState() * 177);
        FlagArrowMC.SetDisplayInfo(info);
        info.Clear();
    }
    else if (bFlagMCVisible & !penv->IsHUDPlayerCapturingObjective())
    {

```

```

        // If the flag indicator is still visible and the player is
        // no longer capturing an objective, play the fade-out animation
        FlagMC.GotoAndPlay("off");
        bFlagMCVisible = false;
    }
}

flagMC.GotoAndPlay("on")    flagMC    Visible
flagMC.GotoAndPlay("off")    flagMC    11-20    20
    1    Flash    ActionScript    C++    1
stop();    1    1
    Alpha    0

arrow    Gfx::Value::SetDisplayInfo\(\)    Gfx::Value::DisplayInfo
        CaptureState    X

```

4.4

```

Reticule    HUDKit fla    1    reticule
    left    right    bottom    top

PlayerFire

```



8 reticule

```

        FxHUDView::UpdateEvents()    FxHUDView::OnEvent()
PlayerFire    ReticuleHeat    ReticuleFiringOffset    FxHUDEvent

// If the player is firing his/her weapon, update the reticule's heat.
// The reticule elements are updated in FxHUDView::UpdateEvents() based on
// the ReticuleHeat and the ReticuleFiringOffset.
case FxHUDEvent::EVT_PlayerFire:
{
    if (ReticuleHeat < 8)
        ReticuleHeat += 2.5f;
}

```

```

    ReticleFiringOffset = 2;
}

```

```

FxHUDView::UpdateEvents()

```

```

    SetDisplayInfo()

```

```

    0, 0

```

```

    X    Y

```

```

Gfx::Value::DisplayInfo

```

```

    SETDISP(7,16,23,0)2B279340pGLX

```

```

    top  bottom  left  right

```

```

    X

```

```

    Y

```

```

    6

```

```

    -6

```

```

    SetDisplayInfo()

```

```

    DisplayInfo

```

```

    5

```

```

    L

```

```

    "

```

```

    ?

```

```

    E

```

```

    ?

```

```

    G

```

```

    ?

```

```

    ?

```

4.5

dirHit HUDKit.fla 1 *dirHit* *dirHit*

tl l bl b br r tr t

Alpha

a)

b)



9 *dirHit*

~1

Flash

Classic Tween Alpha

Gfx::Value::GotoAndPlay("on")

```
void FxHUDView::OnEvent( FxHUDEvent* pevent )
{
    switch ( pevent->GetType() )
    {
        // If the Damage Event (Player was hit) is fired, show the appropriate
        // directional hit indicator.
        // We can use GotoAndPlay("on") to play the fade-in animation. It will
        // fade out on its own after ~1 second (this animation is setup on the
        // Flash timeline)
        case FxHUDEvent::EVT_Damage:
        {
            FxHUDDamageEvent* peventDamage = (FxHUDDamageEvent*)pevent;
            float dir = peventDamage->GetDirection();
            if (dir > 0)
            {
                if (dir > 90)
                {
                    if (dir > 135)
                        DirMC_B.GotoAndPlay("on");
                    else
                        DirMC_BR.GotoAndPlay("on");
                }
            }
            else
            {
                if (dir < -90)
                {
                    if (dir < -135)
                        DirMC_L.GotoAndPlay("on");
                    else
                        DirMC_LL.GotoAndPlay("on");
                }
            }
        }
    }
}
```

```

        {
            if (dir > 45)
                DirMC_TR.GotoAndPlay("on");
            else
                DirMC_T.GotoAndPlay("on");
        }
    }
else
{
    if (dir < -90)
    {
        if (dir < -135)
            DirMC_B.GotoAndPlay("on");
        else
            DirMC_BL.GotoAndPlay("on");
    }
    else
    {
        if (dir < -45)
            DirMC_L.GotoAndPlay("on");
        else
            DirMC_TL.GotoAndPlay("on");
    }
}
break;
}
}

```

4.6

expBar HUDKit.fla Scene 1 *expBar*
 expBar *exp* MovieClip

rank HUDKit.fla Scene 1 *rank* *rank* 18 Keyframe
 Keyframe



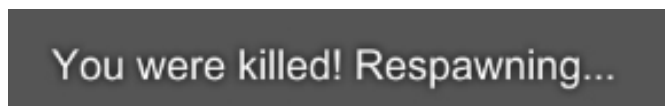
10 expBar rank

```
void FxHUDView::UpdatePlayerEXP(FxHUDEnvironment *penv)
{
    FxHUDPlayer* pplayer = penv->GetHUDPlayer();
    if (pplayer->GetLevelXP() != State.Exp)
    {
        Value::DisplayInfo info;
        // Update the rank icon. Each rank icon is on a different frame of
        // the rank movieClip.
        RankMC.GotoAndStop(UInt(pplayer->GetRank() + 1));
        info.SetScale(pplayer->GetLevelXP() * 100, 100); // EXP
                                                         //
        ExpBarMC.SetDisplayInfo(info);
    }
}
```

Gfx::Value::GotoAndStop() X X + 1 0 *rank*

4.7

centerTextMC HUDKit.fla 1 *centerText*
 centerText



5 centerTextMC

<i>centerTextMC</i>	<i>centerText</i>	<i>textField</i>
	<i>centerTextMC</i>	"on5"
5	"on"	3
1	<i>centerTextMC</i>	Alpha 0
		"on" "on5"

```
TextFieldMC.SetText(RespawnText); // Set the text field of the Center Text.
CenterTextMC.GotoAndPlay("on"); // Play the fade-in animation, it will fade on its
                                // own after ~3 sec.
```

<i>centerTextMC</i>	<i>textField</i>	<i>GotoAndPlay("on")</i>
Classic Tween	~3	

4.8

<i>popup</i>	HUDKit.fla	1	<i>popup</i>
	3		<i>popup</i>
3-5	C++		



6 popup

<i>Popup</i>		Classic Tweens	<i>popup</i>
<i>popup</i>	4	<i>actions</i>	<i>popupNumber</i> <i>popupText</i> <i>popupBG</i> <i>popupBG</i>
		<i>popupText</i>	<i>popupNumber</i> <i>textField</i>
"on"		<i>GotoAndPlay()</i>	
	1	Alpha	0

```
// If a KillStreak event is fired, display the kill streak pop-up
case FxHUDEvent::EVT_KillStreak:
{
    FxHUDKillStreakEvent* peventKS = (FxHUDKillStreakEvent*)pevent;

    // Format the number of kills
    String text;
    Format(text, "{0}", peventKS->GetSrcKillStreak());
    PTextFieldMC.SetText(text); // PopUp
```

```

        // Play the fade-in animation, it will fade on its own after ~4 sec.
        PopupMC.GotoAndPlay("on");
    }
    break;

```

	FxHUDEvent	KillStreak		
C++		popupText's textField	PTextFieldMC	GotoAndPlay("on")
popup				

4.9

log	HUDKit fla	Scene 1	log	HUD
				~3



13 log logMessage

logMessage		3
logMessage	C++	htmlText textField

```

        FxHUDMessage::Movieclip* FxHUDLog::GetUnusedMessageMovieclip()
        LogMessage MovieClip          Log

```

```

FxHUDMessage::Movieclip* FxHUDLog::GetUnusedMessageMovieclip()
{
    if (MessageMCs.IsEmpty())
    {
        // Request a new line in the SWF
        Value logline;
        LogMC.AttachMovie(&logline, "LogMessage", "logMessage"+NumMessages);
        FxHUDMessage::Movieclip* pmc = new FxHUDMessage::Movieclip(logline);
        MessageMCs.PushBack(pmc); // Add new Message MC to list of unused
                                   message movieClips.
    }
}

```

```

    }
    FxHUDMessage::Movieclip* pmc = MessageMCs.GetFirst(); // Use an unused Message
                                                         movieClip.

    pmc->SetVisible(true);
    MessageMCs.Remove(pmc);
    return pmc;
}

```

	C++	C++	Level Up	
"You are now a [Rank Icon] [Rank Name]"	[	]	[	htmlText

```

void FxHUDLog::Process(FxHUDEvent *pevent)
{
    String msg;
    switch (pevent->GetType())
    {
        case FxHUDEvent::EVT_LevelUp:
        {
            FxHUDLevelUpEvent* e = (FxHUDLevelUpEvent*)pevent;
            Format(msg, "You are now a <img src='rank{1}'/> {0}!",
                e->GetNewRankName(), e->GetNewRank());
        }
        break;
    }
}

```

```

FxHUDMessage::Movieclip* pmc = GetUnusedMessageMovieclip();
FxHUDMessage* m = new FxHUDMessage(msg, pmc); // 3
Log.PushBack(m);
NumMessages++;

```

HUD Update

```

void FxHUDLog::Update()
{
    SF_ASSERT(LogMC.IsDisplayObject());

    Double rowHeight = 30.f;

    // Tick the message lifetimes
    Double yoff = 0;
    FxHUDMessage* data = Log.GetFirst();
    while (!Log.IsNull(data))
    {
        FxHUDMessage* next = Log.GetNext(data);
        if (data->IsAlive())
        {

```

```

        // Layout mgmt and animation
        Value::DisplayInfo info;
        info.SetY(yoff);
        data->GetMovieclip()->GetRef().SetDisplayInfo(info);
    }
    data = next;
    yoff += rowHeight;
}

// Layout management and animation
Value::DisplayInfo info;
info.SetY(LogOriginalY - (NumMessages * rowHeight));
LogMC.SetDisplayInfo(info);

// Remove dead messages
data = Log.GetFirst();
while (!Log.IsNull(data))
{
    FxHUDMessage* next = Log.GetNext(data);
    if (!data->IsAlive())
    {
        Log.Remove(data);
        NumMessages--;

        FxHUDMessage::Movieclip* pmc = data->GetMovieclip();
        pmc->SetVisible(false);
        MessageMCs.PushBack(pmc);

        delete data;
    }
    data = next;
}
}

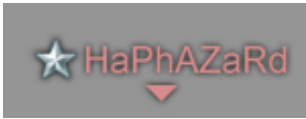
Log                                FxHUDMessage                                0                                FxHUDMessage
                                Log                                FxHUDMessage MovieClips                                MessageMCs
                                FxHUDMessage                                Gfx::Value::DisplayInfo.SetY()
Gfx::Value::SetDisplayInfo()                                MessageMC

```

4.10

billboard_container HUDKit.fla Scene 1 billboard 3D 3D

(Frustum)



7 billboard_enemy

UI

2D

Gfx::Value::SetDisplayInfo()

```
MovieClip (billboard_friendly, billboard_enemy)
billboard_label_(friendly/enemy)                billboard_label_*
textField billboard_friendly's "Show" Keyframe  billboard_friendly
Classic Tween                                   "Show"
```

C++

FxHUDView::BillboardCache::GetUnusedBillboardMovieclip :

```
if (UnusedBillboards.IsEmpty())
{
    Value::DisplayInfo info(false);

    Value billboard, temp;
    String instanceName;
    Format(instanceName, "{0}{1}", BillboardPrefix, BillboardCount++);
    CanvasMC.AttachMovie(&billboard, SymbolLinkage, instanceName);

    BillboardMC* pbb = new BillboardMC();
    pbb->Billboard = billboard;
    billboard.GetMember("label", &temp);
    temp.GetMember("textField", &temp);
    pbb->Textfield = temp;

    pbb->Billboard.SetDisplayInfo(info);
    UnusedBillboards.PushBack(pbb);
}
```

BillboardCache

CanvasMC

BillboardCache

billboard_enemy/billboard_friendly

CanvasMC.AttachMovie()

```
Billboard                                UpdateBillboards()  BeginProcessing()  EndProcessing()
GetUnusedBillboardMovieclip()                                BeginProcessing()
EndProcessing()
```

UpdateBillboards()

```
// Process friendlies
// Friendly billboards will always show names
```

```
BillboardMC* pbb = NULL;
```

```
FriendlyBillboards.BeginProcessing();
```

```
for (unsigned i=0; i < friendlies.GetSize(); i++)
```

```
{
```

```
    info.Clear();
```

```
    if (FriendlyBillboards.GetUnusedBillboardMovieclip(friendlies[i].pPlayer,
                                                         &pbb))
```

```
    {
```

```
        info.SetVisible(true);
```

```
        pbb->Billboard.GotoAndPlay("show");
```

```
        // Load player info
```

```
        String title;
```

```
        Format(title,
```

```
                "<img src='rank{0}' width='20' height='20' align='baseline'
```

```
                vspace='-7' /> {1}",
```

```
                friendlies[i].pPlayer->GetRank()+1,
```

```
                friendlies[i].pPlayer->GetName());
```

```
        pbb->Textfield.SetTextHTML(title);
```

```
    }
```

```
    info.SetX(friendlies[i].X);
```

```
    info.SetY(friendlies[i].Y);
```

```
    info.SetScale(friendlies[i].Scale, friendlies[i].Scale);
```

```
    pbb->Billboard.SetDisplayInfo(info);
```

```
}
```

```
FriendlyBillboards.EndProcessing();
```

```
UpdateBillboards()
```

```
GotoAndPlay("show")
```

```
GFx::Value::SetDisplayInfo()
```

```
MovieClip's X Y
```

```
GFx::Value::SetText()
```

```
textField      htmlText  
src='rank{0}' HTML  
>GetRank()+1)  
{1}
```

img

```
+ 1 (friendlies[i].pPlayer -  
vspace  
(friendlies[i].pPlayer->GetName())
```

5

(Minimap) C++

Apps\Kits\HUD\

- **FxMinimap.h**
- **FxMinimapView.h/.cpp**

5.1.1

- **FxMinimapIconType** ID
 - **FxMinimapIcon**
 - **FxMinimapIconCache** /
- MinimapIconFactoryBase
- MovieClips
Update()

textField

o *PlayerIcon*

o *FlagIcon*

o *ObjectiveIcon*

o *WaypointIcon*

- **FxMinimapView**

UpdateView()

PlayerIcon::Update()

FxMinimapView.cpp

116

Direct

Access API

MovieClip

MovieClip

SetDisplayInfo

Alpha

SetMember()

Gfx::Value::SetMember

Gfx::Movie::SetVariable

```
virtual void Update(FxMinimapView* pview, FxMinimapEntity* pentity,
                    float distSquared)
```

```
{
```

```
    SF_UNUSED(distSquared);
```

```
    bool hasIcon = (pentity->GetIconRef() != NULL);
```

```
    PointF pt = pentity->GetPhysicalPosition();
```

```
    bool showDir = pview->IsShowingPlayerDir();
```

```
    // If entity did not have an icon attached before, then wait for the
```

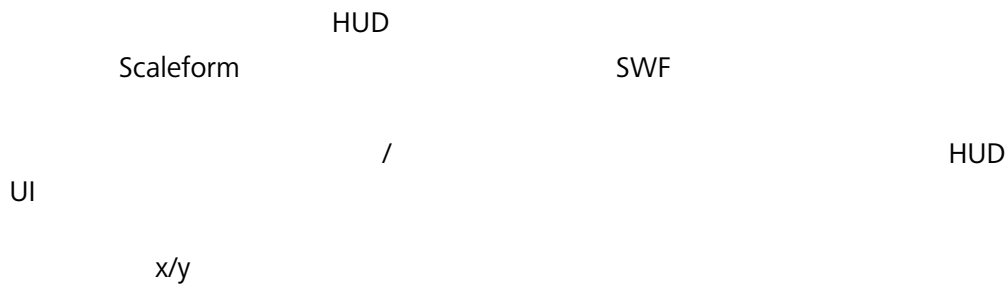
```

// next update to set the state data. picon is most probably not initialized
// (the movie needs to advance after creation)
if (hasIcon && (State != (int)showDir))
{
    // state 0: no arrow, state 1: show arrow
    Value frame(Double(showDir ? 2.0 : 1.0));
    MovieClip.Invoke("gotoAndStop", NULL, &frame, 1);
    State = (int)showDir;
}

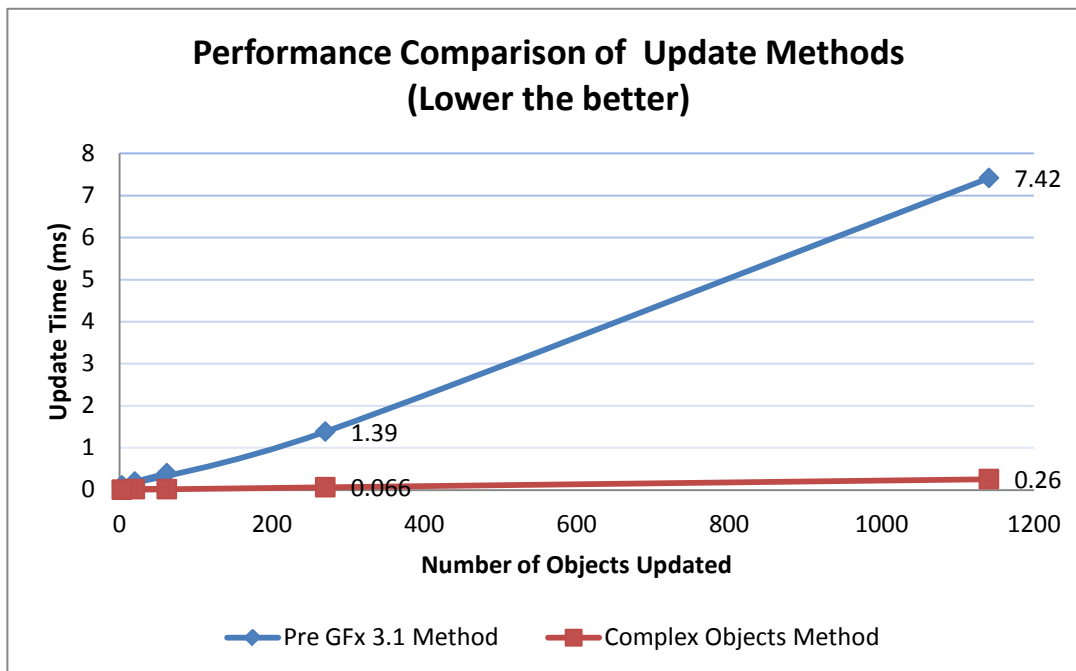
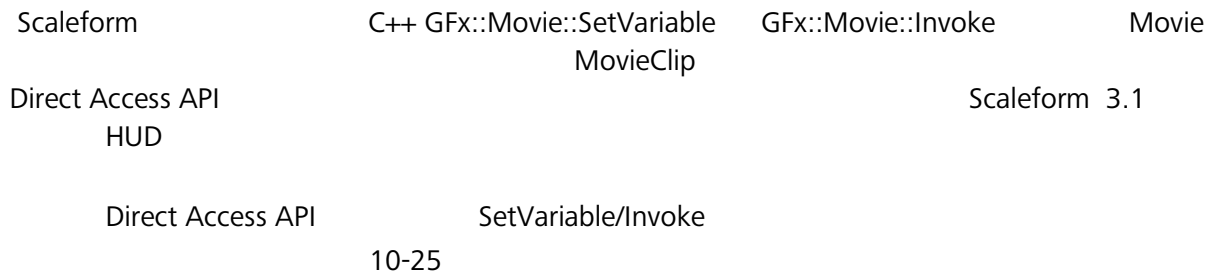
pt = pview->GetIconTransform().Transform(pt);
Value::DisplayInfo info;
if (State)
{
    info.SetRotation(pview->IsCompassLocked() ?
        (pentity->GetDirection() + 90) : (pentity->GetDirection() -
        pview->GetPlayerDirection()));
}
info.SetPosition(pt.x, pt.y);
MovieClip.SetDisplayInfo(info);
}

```

6



6.1



8 Direct Access API SetVariable/Invoke

Direct Access
– HUD UI

Scaleform	15	Flash HUD	HUD	
	FPS			
	1148	0.018	0.512	0.017